# 5 Database

## 5.1 Introduction

### 5.1.1 What is a database

A database can be defined as a collection of related data items within a specific business. DBMS are sotwares that define, create, use and maintain a database, if we combine a DBMS with a database we get a database system The majority of DBMS softwares rely on SQL systems.

### 5.1.2 Conceptual Modelling of a database

To work with databases we need to model them to understand how they work. Modelling means representing reality in a scaled version. The same with databases we connect information to represent how relationships in the world exist. Based on the business needings we design the database in a logical fashion and we translate it to a our language of choice. The step is passing from conceptual design to logical design. The first is DBMS independent while the second is DBMS dependent. The process is:

1. Requirement collection and analysis
2. Conceptual design
3. Logical design
4. Physical design

An SQL database is based on entities and relationships. Entities are business concepts with an unambiguous meaning to a particular set of users. Entities have certain characteristics called attributes. The rows are the entities (also called instances). The columns are the attributes. An association between one or more entities is called relationship, it defines how tables are related to each other.

Another useful concept is the one of cardinality, how many instances of the entities can be associated with each other.

We can have

- 1 to 1
- 1 to many
- Many to many
- None to many
- 1 to None

### 5.1.3 Normalization

An important step in the design is the one related to data normalization, we want to avoid redundant data. We split the tables into smaller tables to avoid anomalies during insertion, deletion or updates. The procedure follows a step by step transformation. The database is more complex from the structural point of view but can be easier to manage.

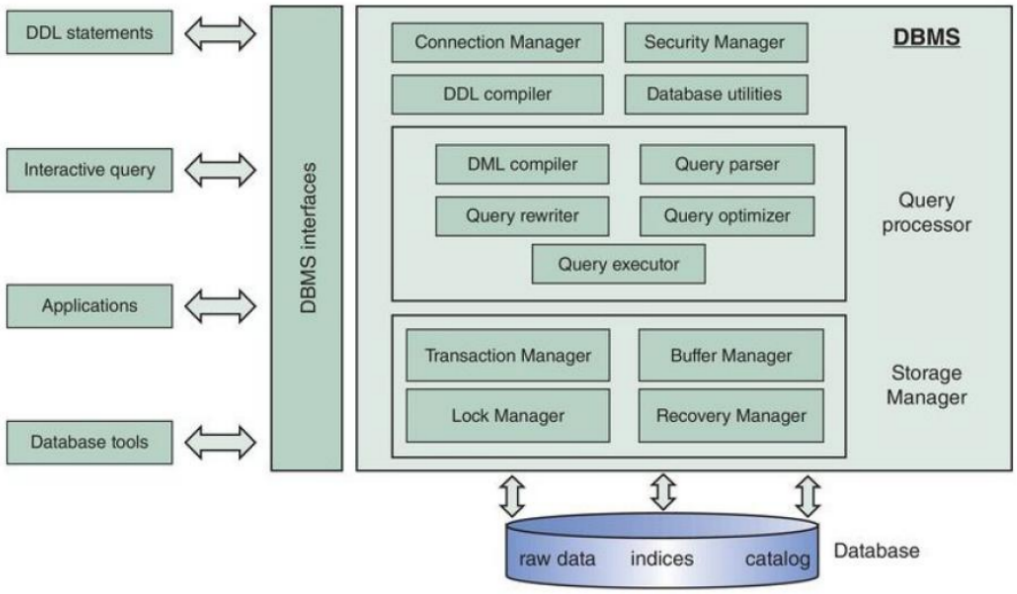
We have 3 possible normal forms.

- First NF →Each cell in the table can have only one value, never a list of values
- Second NF →An entity type is in 2NF when it is in 1NF and when all of its non-key attributes are functionally dependent on its primary key. Features should be fully dependent on the entire primary key
- Third NF →An entity type is in 3NF when it is in 2NF, and non-key attributes are directly (non-transitively) dependent on the primary key. In other words: non-prime attributes must be functionally dependent on the key(s), but they must not depend on another non-prime attribute.

## 5.2 Database modelling

### 5.2.1 Architecture

All DBMS follow a certain structure made by certain elements



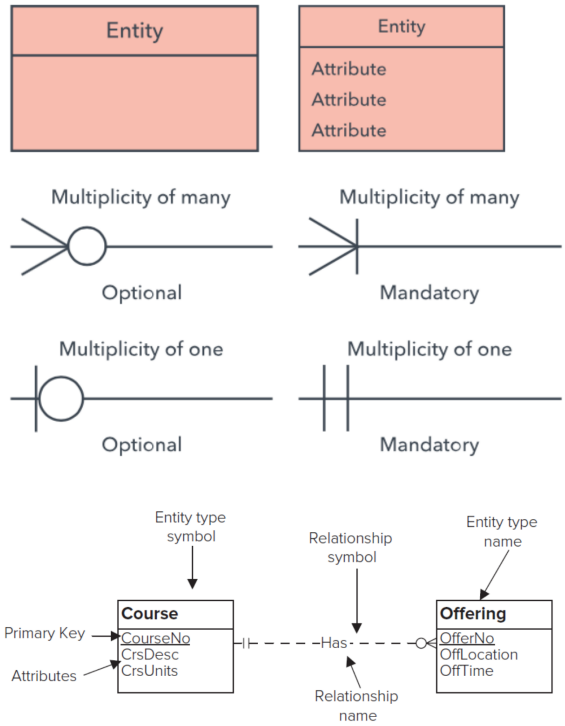

- **Connection manager** →provides facilities to set-up a database connection. It can be set-up locally or through a network
- **Security manager**→verifies whether a user has the right privileges to execute the database actions required
- **DDL compiler**→compiles the data definitions specified in DDL. Most relational databases use SQL as their DDL
- **Utilities**→A loading utility (load data from a variety of sources), reorganization utility (reorganizes the data), user management utilities (support the creation of user groups or accounts), etc
- **DML compiler**→compiler compiles the data manipulation statements specified in DML
- **Query parser**→parses the query into an internal representation format that can then be further evaluated by the system. It checks the query for syntactical and semantical correctness
- **Query rewriter**→optimizes the query, independently of the current database state. It simplifies it using a set of predefined rules and heuristics that are DBMS-specific
- **Query optimizer**→optimizes the query based upon the current database state. It can make use of predefined indexes that are part of the internal data model and provide quick access to the data
- **Query executor**→takes care of the actual execution by calling on the storage manager to retrieve the data requested
- **Transaction manager**→supervises the execution of database transactions. Remember, a database transaction is a sequence of read/write operations considered to be an atomic unit
- **Buffer manager**→is responsible for managing the buffer memory of the DBMS. The DBMS checks first the memory when data need to be retrieved. Retrieving data from the buffer is significantly faster than retrieving them from external disk-based storage
- **Lock manager**→essential component for providing concurrency control, which ensures data integrity at all times
- **Recovery manager**→supervises the correct execution of database transactions. It keeps track of all database operations in a logfile and will be called upon to undo actions of aborted transactions or during crash recovery

A DBMS needs to interact with various parties, such as a database designer, a database administrator, an application, or even an end-user (Ex. APIs).

### 5.2.2 ERD

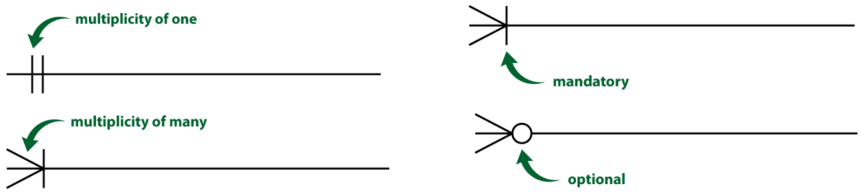
To design our databases we can use an ERD (Entity Relationship Diagram) notation.

Some of them can be Chen, IDEF1X, Bachman, Crow's Foot (we use this), Min-Max or UML.




An entity is represented by a rectangle, with its name on the top. The name is singular. The attribute(s) that uniquely distinguishes an instance of the entity is the identifier.

Relationships have two indicators. The first one refers to the maximum number of times that an instance of one entity can be associated with instances in the related entity. The second one describes the minimum number of times one instance can be related to others. It can be zero or one, and optional or mandatory.




### 5.2.3 From ERD to tables

Relations can be

- One to one
- One to many
- Many to many

Since we are working with tables we need to pass from schemas to tables, we use keys (foreign and primary). We can also create a linking table to model many to many relations.

## 5.3 SQL

### 5.3.1 What is SQL

SQL means Structured Query Language. It was created by IBM in 1974 and can be implemented by each vendor (SQL dialects).

Key characteristics

- Set-oriented and declarative
- Free form
- Case insensitive
- Can be used via prompt or via a program

### 5.3.2 Create a database

To show existing databases

```
SHOW DATABASES;
```

Create database

```
CREATE DATABASE [IF NOT EXISTS] database_name
[CHARACTER SET charset_name]
[COLLATE collation_name]
```

Use Database

```
USE database_name;
```

Drop database

```
DROP DATABASE [IF EXISTS]
database_name;
```

See tables in the database

```
SHOW TABLES;
```

### 5.3.3 Create tables

Create a Table

```
    CREATE TABLE [IF NOT EXISTS] table_name(
col1 data_type(length) [NOT NULL] [DEFAULT value] [AUTO_INCREMENT]
col2 data_type(length) [NOT NULL] [DEFAULT value] [AUTO_INCREMENT]
col3 data_type(length) [NOT NULL] [DEFAULT value] [AUTO_INCREMENT]
) ENGINE=storage_engine;
```

| Data Type | Description |
|---|---|
| CHAR(size) | Holds a fixed length string (can contain letters, numbers, and special characters). The fixed size is specified in parenthesis. Can store up to 255 characters. |
| VARCHAR(size) | Holds a variable length string (can contain letters, numbers, and special characters). The maximum size is specified in parenthesis. Can store up to 255 characters. Note: If you put a greater value than 255 it will be converted to a TEXT type. |
| TEXT | Holds a string with a maximum length of 65,535 characters. |
| INT(size) | Integer. Range: -2,147,483,648 to 2,147,483,647 (normal), 0 to 4,294,967,295 (UNSIGNED). Maximum number of digits may be specified in parenthesis. |
| FLOAT(size,d) | Small number with floating decimal point. Maximum number of digits may be specified in `size`. Maximum digits to the right of the decimal point are specified in `d`. |
| DOUBLE(size,d) | Large number with floating decimal point. Maximum number of digits may be specified in `size`. Maximum digits to the right of the decimal point are specified in `d`. |
| DATE() | A date. Format: YYYY-MM-DD. Supported range: '1000-01-01' to '9999-12-31'. |
| YEAR() | A year in two-digit or four-digit format. Values allowed in four-digit format: 1901 to 2155. Values allowed in two-digit format: 70 to 69, representing years from 1970 to 2069. |

Table 13: Common SQL Data Types

Primary keys

```
CREATE TABLE [IF NOT EXISTS] table_name(
col1 data_type(length) [PRIMARY KEY]
col2 data_type(length) [PRIMARY KEY]
col3 data_type(length) [PRIMARY KEY]
);

CREATE TABLE [IF NOT EXISTS] table_name(
col1 data_type(length)
col2 data_type(length)
col3 data_type(length)
PRIMARY KEY(col1, col2, ...)
);

CREATE TABLE Person (
pid int AUTO_INCREMENT NOT NULL,
firstname varchar(25) NOT NULL,
surname varchar(25) NOT NULL,
PRIMARY KEY (pid)
);

CREATE TABLE [IF NOT EXISTS] table_name(
col1 data_type(length)
PRIMARY KEY(col1, col2, ...)
CONSTRAINT constraint_name  \ra define constraint name for the foreign key
FOREIGN KEY (columns)
REFERENCES parent_table (columns)
ON DELETE action    \ra CASCADE, SET NULL, NO ACTION, RESTRICT
```

```
ON UPDATE action    \ra CASCADE, SET NULL, NO ACTION, RESTRICT
);
```

Foreign key examples

```
CREATE TABLE category(
categoryId INT AUTO_INCREMENT PRIMARY KEY,
categoryName VARCHAR(100) NOT NULL);

CREATE TABLE products(
productId INT AUTO_INCREMENT PRIMARY KEY,
productName varchar(100) not null,
categoryId INT NOT NULL,
CONSTRAINT fk_category
FOREIGN KEY (categoryId)
REFERENCES category(categoryId)
ON UPDATE SET NULL
ON DELETE SET NULL );
```

Drop a table

```
DROP TABLE [IF EXISTS] table_name [, table_name]
```

### 5.3.4 CRUD operations

CRUD operations refere to those operations that are meant to Create, Read, Update and Delete data in our database.

To insert data

```
    INSERT INTO table_name(c1, c2,...)
    VALUES (v1,v2,...);
```

To read data

```
    SELECT column_1, column_2,\dots
    FROM table_1
    [INNER|LEFT|RIGHT] JOIN table_2 ON condition
    WHERE conditions
    GROUP BY column_1
    HAVING group_conditions
    ORDER BY column_1
    LIMIT offset, length;
```

To update our data

```
UPDATE [LOW_PRIORITY] [IGNORE] table_name
SET column_name1 = expr1,
column_name2 = expr2,
[WHERE condition];
```

To delete data

```
DELETE FROM table_name
WHERE condition;
```

### 5.3.5 SQL clauses and operators

The main clauses and operators that we have in SQL are

- LIMIT
- DISTINCT
- ORDER BY
- BETWEEN

NOT LIKE

LIMIT is used in the select statement to costrain the number of rows to return

```
SELECT select_list
FROM table\_name
LIMIT [offset] row_count;
```

Offset specifies the offset of the first row to return (from 0) while row_count the maximum number of rows to return

LIMIT will remove duplicates

```
SELECT DISTINCT e.first_name FROM employees AS e LIMIT 10;
```

The BETWEEN operator is a logical one that we can use to specify a value within or not a range

```
SELECT e.emp_no, e.first_name, e.last_name, e.hire_date
FROM employees AS e
WHERE e.hire_date BETWEEN '1999-01-01' AND '1999-12-31';
```

The LIKE operator is a logical operator that tests whether a string contains a specified pattern or not

```
SELECT e.first_name, e.last_name
FROM employees AS e
WHERE e.last_name LIKE 'T%';


SELECT e.first_name, e.last_name
FROM employees AS e
WHERE e.last_name NOT LIKE 'Tr%';
```

We can use the keyword AS to refere in a different way

```
table_name AS table_alias

SELECT
[column_1 | expression] AS descriptive_name
FROM table_name;

SELECT
e.firstName, e.lastName
FROM employees e
ORDER BY e.firstName;

SELECT
CONCAT (lastName, ' ', firstname) AS Full_name
FROM employees;
```

The IN operator allows you to determine if a specified value matches any value in a set of values or returned by a subquery.

```
SELECT column1,column2,...
FROM table_name
WHERE column_n IN (value1,value2,...);
```

The SELECT statement does not provide the result set sorted. To sort the result set, you add the ORDER BY clause to the SELECT statement

```
SELECT select_list
FROM table_name
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC], ...;
```

### 5.3.6 SQL joins

There are four basic types of SQL joins: inner, left, right, and full.

INNER

```
SELECT column_list
FROM table_A
INNER JOIN table_B ON join_condition;

SELECT column_list
FROM table_A
INNER JOIN table_B USING (column_name);

SELECT column_list
FROM table_A A, table_B B
WHERE A.column_name = B.column_name;

SELECT column_list
FROM Table_A A
INNER JOIN Table_B B
ON A.Key = B.Key;
```

LEFT

```
SELECT column_list
FROM Table_A A
LEFT JOIN Table_B B
ON A.Key = B.Key;
```

RIGHT

```
ELECT column_list
FROM Table_A A
RIGHT JOIN Table_B B
ON A.Key = B.Key;
```

OUTER/FULL

```
SELECT * FROM t1
LEFT JOIN t2 ON t1.id = t2.id
UNION ALL
SELECT * FROM t1
RIGHT JOIN t2 ON t1.id = t2.id
```

Left excluding

```
SELECT column_list
FROM Table_A A
```

```
LEFT JOIN Table_B B
ON A.Key = B.Key
WHERE B.Key IS NULL;
```

Right excluding

```
SELECT column_list
FROM Table_A A
RIGHT JOIN Table_B B
ON A.Key = B.Key
WHERE A.Key IS NULL;
```

Self

```
SELECT m.lastName AS Manager, e.lastName AS 'Direct report'
FROM employees e
INNER JOIN employees m ON m.employeeNumber = e.reportsTo;
```

### 5.3.7 Functions

Functions in SQL can be

- Aggregate →Performs a calculation on multiple values and returns a single value. Can be AVG(), MAX(), MIN(), STDEV(), COUNT() or SUM(). COUNT() has three main forms
  1. COUNT(*) →returns the number of rows including duplicates, non-NULL and NULL rows.
  2. COUNT(expression) →returns the number of rows with non NULL values
  3. COUNT(DISTINCT expression) →returns the number of distinct rows that do not contain NULL values

```
SELECT count(employee_id) as employees
FROM employee
WHERE department_id = 50;
```

- Math →mathematical computations. Can be MOD(), ROUND(), TRUNCATE(), SIN(n), SQRT(n), RAND(), LOG(n) or POW()

```
SELECT round(salary, 2)
FROM employee;
```

- Date →make computations on date values. Can be CURDATE()/CURRENT_DATE(), DATEDIFF(), DAY(), DAYOFWEEK(), NOW(), MONTH(), TIMEDIFF() or YEAR().
- String →work on strings. Can be CONCAT(), LENGTH(), LOWER(), REPLACE(), SUBSTRING(), TRIM(), UPPER() or INSTR()

### 5.3.8 Group By

The GROUP BY clause tries to take a set of rows and summarize them by values. It returns one row for each group. Is used with SUM, AVG, MAX, MIN and COUNT.

```
SELECT c1, c2, ..., cn, aggregate_function(ci)
FROM table_name
WHERE where_conditions
GROUP BY c1, c2, ..., cn;
```

### 5.3.9 Having

The HAVING clause is used in the SELECT statement to specify filter conditions for a group of rows or aggregates. Oten used with GROUP BY to filter groups based on a specific condition. HAVING applies a filter condition to each group of rows, while the WHERE does that to each individual row.

```
SELECT select_list
FROM table_name
WHERE search_condition
GROUP BY group_by_expression
HAVING group_condition
```

### 5.3.10 Views

A VIEW is defined by means of an SQL query and its content is generated upon invocation of the view by an application or other query. Queries can be run against the view. A view will not physically store the data. Each time a view is queried the underlying query called view definition is executed. We create views to do queries that we might want to do frequently.

```
CREATE[OR REPLACE] VIEW [db_name.]view_name[(column_list)]
AS select-statement
```

### 5.3.11 Triggers

A TRIGGER is a piece of SQL code made of declarative and/or procedural instructions and stored in the catalog of the RDBMS. Is automatically runned whenever a specific event (INSERT, UPDATE, DELETE) occurs and a specific condition is evaluated as true.

```
CREATE TRIGGER trigger_name trigger_time trigger_event
ON table_name
FOR EACH ROW
BEGIN what trigger does
END;
```

Where trigger time can be BEFORE or AFTER.
Where trigger event can be INSET, UPDATE or DELETE.

Advantages

- Aumatization and verification in case of specific events or situations
- Can model more rules without change the code
- Can assign default values for new tuples
- Synchronic updates in case of data replication
- Automatic auditing and logging
- Automatic exporting of data

Disadvantages

- Hidden, difficult to manage
- Cascade effects
- Uncertain outcomes
- Deadlock situations
- Debugging complexities
- Maintainability problems

## 5.4 Query Optimization

### 5.4.1 The problem

Users can complain that applications are slow. One reason can be that the queries in the SQL database are slow and not optimized.

User's feeling about response time

- 0.1 second →feels lika instantaneous
- 1 second →limit to that feeling
- 10 seconds →the user will do something else

A query of 3.19 seconds can be considered as slow, we need to understand if it has problems. We can do that by using a QEP (Query Execution Plan). A query plan is a list of instructions that the database needs in order to execute a query on the data. The query optimizer will generate multiple query plans for a single query and will select the most efficient one.

```
SELECT *
FROM tools
WHERE name="Screwdriver";
```

vs

```
SELECT *
FROM tools
WHERE id = 3;
```

Both are returning the same results but the second one because the index is usually unique and comparisons between strings is slower.

### 5.4.2 EXPLAIN

The EXPLAIN command is used to show query plans. It doesn't run the actual SQL statement (except subqueries). It doesn't provide any tuning recommendation but information that can be used to do that. For example we can use a newly created index with an alter statement.

How to identify problems using EXPLAIN

- No index used (NULL in the key column)
- A lot of rows are processed (rows column)
- A lot of indexes are evaluated (possible_keys column)

Remember

- We can optimize queries in many ways
- The generated query plan can change depending o many factors
- For UPDATE and DELETE statements we need to rewrite the query as a SELECT statement to confirm index usage

### 5.4.3 Indexes

We need to be sure that our indexes make sense. An index is a structure that holds the field the index is sorting and a pointer from each record to their corresponding record in the original table where the data is stored. The id is incrementing based on the order in which the data was added. To fasten the search the data are organized like a binary tree based on the middle entry. Indexing makes columns faster to query by creating pointers to where data is stored within a database.

Index cardinality (amount of unique values)

- Multiple different indexes make MYSQL try to identify the most effective index for the query.
- The optimizer chooses an index based on the estimated cost to do the least amount of work
- Index cardinality can be used to to that

Normally indexes are created when the table is created. After we can do that using

```
CREATE INDEX index_name ON table_name
(column_list)
```

### 5.4.4 Data types

Even data types can impact performance based on their structures. Usually the simpler teh datatype is the better it performs. The best way to do this is to use a data type that is native for the data that we need to use

- tinyint
- int
- bigint
- json
- longtext
- longblob
- decimal
- float
- double
- enum

### 5.4.5 Tips

1. Define SELECT fields instead of SELECT *
2. Avoid SELECT DISTINCT if possible
3. Use WHERE instead of HAVING
4. Use WILDCARDS at the end of the phrase
5. Use LIMIT
6. Run queries during off-peak times
7. Index the queries properly →the column is queried frequently, foreign keys to other tables and a unique key exists on the columns

## 5.5 CAP theorem and NOSQL databases

### 5.5.1 ACID properties

Transactions are collections of actions that make consistent transformations of systems preserving consistency

- A →atomicity, the changes must be fully done
- C →consistency, any database transaction must change affected data only in allowed ways
- I →isolation, how and when the changes made by one operation become visible to others
- D →durability, transactions that have been commited must survive permanently

RDBMs must be ACID compliant. New data or modifications are noit accepted unless they comply with the given schema in terms of data, integrity etc. In big databases we need flexibility since we have a lot of data variety. Also new functionalities need all the elements to support the new structure.

### 5.5.2 NOSQL

Since we might have multiple queries together we can scale our server both vertically or horizontally

- Vertically →extending storage capacity and/or CPU power of the database server
- Horizontally →multiple DBMS servers being arranged in a cluster

RDBMs can have problems in horizontally scaling. NOSQL databases solve this by using formats other than tabular relations. They aims at near linear horizontal scalability distributing data on clusters of database nodes for improving performance and availability. Also we have eventual consistency: the data will become consistent at a certain point. The languages can vary from different languages

| | Relational Databases | NoSQL Databases |
|---|---|---|
| Data paradigm | Relational tables | Key–value (tuple) based<br>Document based<br>Column based<br>Graph based<br>XML, object based<br>Others: time series, probabilistic, etc. |
| Distribution | Single-node and distributed | Mainly distributed |
| Scalability | Vertical scaling, harder to scale horizontally | Easy to scale horizontally, easy data replication |
| Openness | Closed and open source | Mainly open source |
| Schema role | Schema-driven | Mainly schema-free or flexible schema |
| Query language | SQL as query language | No or simple querying facilities, or special-purpose languages |
| Transaction mechanism | ACID: Atomicity, Consistency, Isolation, Durability | BASE: Basically Available, Soft state, Eventual consistency |
| Feature set | Many features (triggers, views, stored procedures, etc.) | Simple API |
| Data volume | Capable of handling normal-sized datasets | Capable of handling huge amounts of data and/or very high frequencies of read/write requests |

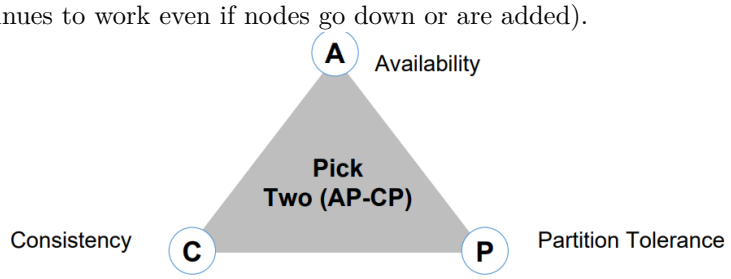
NOSQL databases as rule of thumb are not acid compliant than can be problematic for critical trnasactions.

NOSQL relies on BaSE model

- Basically available →Guarantees the availability of the data . There will be a response to any request (can be failure too).
- Soft state →The system can change over time, even without receiving input (since nodes continue to update each other)
- Eventual consistency →The system will become consistent over time but might not be consistent at a particular moment

### 5.5.3 CAP theorem

A distributed computer system, such as the one we've looked at above, cannot guarantee the following three properties simultaneously: consistency (all nodes see the same data simultaneously); availability (guarantees that every request receives a response indicating a success or failure result); and partition tolerance (the system continues to work even if nodes go down or are added).




We have to choose between 2 of those points

- CA →Vertical scalability, SQL
- P →horizontal scalability